MOUNT KENYA UNIVERSITY

School of Computing and Informatics

BIT3102: Event Driven Programming

Group Assignment: ASP.NET Core MVC Responsibilities and Request Flow

Group 1

Name	Registration Number
Abdi Mulki Issack	BSCCS/2025/39591
Issa Bishar	BSCCS/2025/50654
Joshua Samuel	BSCCS/2025/40065
Abdisamad Mohamed	BSCCS/2025/39474
Kanyi Chapline	BSCCS/2025/45195
Ariel Boutcher	BSCCS/2025/40350
Anitah Wairimu	BSCCS/2024/52366
Ng'ang'a Lewis	BSCCS/2025/58918
Aristote Alphonse	BSCCS/2025/40882
Joyce Wanjiku	BIT/2023/64001
Fridah Deraso	BSCCS/2025/35619

1 (a) Roles of Model, View and Controller [5 marks]

ASP.NET Core MVC is built on the Model–View–Controller pattern. The pattern divides a web application into three cooperating parts so that data, presentation and request handling remain clearly separated. This separation makes the application easier to develop, test and maintain.

Model

The Model represents the data of the application and the business rules that apply to that data. In ASP.NET Core it is normally implemented as one or more plain C# classes (sometimes called POCOs — Plain Old CLR Objects).

A model class typically contains:

- Properties that describe the entity (for example `Id`, `RegNumber`, `FullName`, `Programme` for a student).
- Optional validation attributes such as `[Required]` or `[StringLength]`.
- In larger systems, related domain logic that belongs with the data itself.

The model does **not**:

- Generate HTML or decide how data is displayed.
- Read HTTP headers or route values directly.
- Know which view will be used to render it.

In the accompanying demo the model is the `Student` class. It only carries student information; all presentation decisions are left to the view and all request handling is left to the controller.

View

The View is responsible for the user interface. In ASP.NET Core MVC a view is usually a Razor file with a `.cshtml` extension. Razor allows HTML to be combined with C# expressions so that dynamic data can be inserted into the page.

A view:

- Receives data from the controller, most often as a model object (`@model Student`).
- Uses that data to produce HTML (and sometimes CSS or client-side script).
- Should remain focused on presentation — layout, labels, formatting and simple display logic.

A view should **not** contain:

- Database queries or file access.
- Complex business rules (for example fee calculations or admission decisions).
- Direct manipulation of HTTP responses beyond normal HTML generation.

Keeping the view thin makes it easier to change the look of the application without touching business logic, and easier to reuse the same data in different presentations (web page, PDF, API, etc.).

Controller

The Controller is the coordinator. It receives the HTTP request, decides what work is required, interacts with models or services, and selects the view that will produce the response.

In ASP.NET Core a controller is a C# class that inherits from `Controller` (or `ControllerBase` for APIs). Public methods that return `IActionResult` are called **actions**. Each action can be reached by a URL through the routing system.

A typical controller action:

1. Reads input from the route, query string or request body (for example an `id` parameter).
2. Calls a service or queries data to obtain or update a model.
3. Performs simple checks (for example returning `NotFound()` when a record does not exist).
4. Returns a result such as `View(model)`, `RedirectToAction(...)`, or `Json(data)`.

The controller should not embed large amounts of HTML and should not become a dumping ground for all application logic. Its role is to translate an HTTP request into the correct application operation and then hand the result to the appropriate view.

How the three work together

When a request arrives, routing selects a controller and action. The action uses models (and often services) to obtain data, then passes that data to a view. The view turns the data into HTML. The browser finally displays the HTML. Each part has one clear job, which is the essence of the MVC pattern.

2 (b) Complete Request Flow [5 marks]

The following is a detailed description of what happens when a browser requests a page from an ASP.NET Core MVC application, using `/Students/Details/7` as a running example.

1. **User action and HTTP request.** The user types a URL or clicks a link. The browser sends an HTTP request to the server. For our example the request line is effectively:

   ```
   GET /Students/Details/7 HTTP/1.1
   ```

   together with headers such as `Host` and `Accept`.

2. **Server receives the request.** The ASP.NET Core web server (Kestrel, often behind a reverse proxy) accepts the TCP connection and parses the HTTP request into an internal `HttpContext` object that the rest of the pipeline can use.

3. **Middleware pipeline.** Before the MVC framework runs, the request passes through a pipeline of middleware components. Common middleware includes exception handling, HTTPS redirection, static file serving, routing, authentication and authorisation. Each middleware can inspect or modify the request, short-circuit the pipeline, or pass control to the next component. Order matters: for example, routing must run before MVC can select a controller.

4. **Routing.** The routing middleware matches the request path against the route templates registered in `Program.cs`. The default conventional template is:

   `{controller=Home}/{action=Index}/{id?}`

   For the path `/Students/Details/7` routing extracts:

   - `controller` = `Students`
   - `action` = `Details`
   - `id` = `7`

   These values are stored in route data and become available to model binding.

5. **Controller creation and action selection.** MVC uses the route data to locate the class `StudentsController` and the method `Details`. It creates an instance of the controller (often with dependencies injected through the constructor) and prepares to call the action.

6. **Model binding.** The framework binds request data to action parameters. In this case the route value `id = 7` is converted to an `int` and supplied as the argument to `Details(int id)`.

7. **Action execution (controller logic).** The `Details` method runs. It is responsible for obtaining the correct data. In a production system this would usually mean calling a service or database layer. In the demo project the controller searches an in-memory list:

```
var student = Students.FirstOrDefault(s => s.Id == id);
if (student == null)
{
    return NotFound();
}
return View(student);
```

   If no student exists with that id, the action returns a 404 response. Otherwise it returns a view result that carries the student object.

8. **View selection and rendering.** `View(student)` tells MVC to render the Razor view `Views/Students/Detail` and to make the student object available as `Model`. The Razor engine executes the view, evaluates expressions such as `@Model.FullName`, and produces a complete HTML document.

9. **HTTP response.** The generated HTML is written into the response body. Status code 200 is used for a successful page. The response travels back through any remaining middleware and is sent to the client over the network.

10. **Browser rendering.** The browser receives the HTML (and any linked CSS or scripts) and displays the student details page to the user.

At every stage the responsibilities remain distinct: routing finds the action, the controller orchestrates work and selects a view, the model carries data, and the view produces markup.

3 (c) MVC Request-Flow Diagram for `/Students/Details/7` [3 marks]

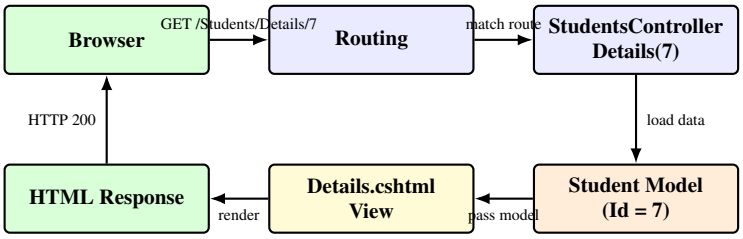


Figure 1: Request flow for `GET /Students/Details/7`

Step-by-step mapping for `/Students/Details/7`

Stage	What happens for this URL
Browser	Sends `GET /Students/Details/7`
Routing	Matches `{controller}/{action}/{id?}` → Controller=`Students`, Action=`Details`, id=`7`
Controller	`StudentsController.Details(7)` executes
Model lookup	Student with `Id == 7` is retrieved (in-memory list in the demo)
View	`Views/Students/Details.cshtml` receives the Student model
Response	HTML page showing that student's details is returned with status 200

Demo project — code explanation

A minimal ASP.NET Core MVC project is supplied with this assignment (`Group1_StudentsMvcDemo`). It implements exactly the flow above and can be run with `dotnet run` (target framework `net10.0`).

1. Model — `Models/Student.cs`

```
namespace StudentsMvcDemo.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string RegNumber { get; set; } = string.Empty;
        public string FullName { get; set; } = string.Empty;
        public string Programme { get; set; } = string.Empty;
    }
}
```

This is a pure data class. It has no dependency on HTTP, HTML or the database. Any part of the application can use it.

2. Controller — `Controllers/StudentsController.cs`

```
public class StudentsController : Controller
{
    private static readonly List<Student> Students = new()
    {
        new Student { Id = 7, RegNumber = "BSCCS/2025/40350",
                      FullName = "Ariel Boutcher",
                      Programme = "BSc Computer Science" },
        // ... other sample students
    };

    public IActionResult Details(int id)
    {
        var student = Students.FirstOrDefault(s => s.Id == id);
        if (student == null)
        {
            return NotFound();
        }
```

```
        return View(student);
    }
}
```

Explanation:

- The static list replaces a database so the demo has zero external dependencies.
- `Details(int id)` is the action invoked for `/Students/Details/7`.
- Model binding converts the route segment `7` into the integer parameter `id`.
- If the student is missing, the action returns HTTP 404 via `NotFound()`.
- If the student is found, `View(student)` selects `Details.cshtml` and passes the object as the view model.

3. View — `Views/Students/Details.cshtml`

```
@model Student
<h2>Student Details</h2>
<dl>
    <dt>Id</dt>
    <dd>@Model.Id</dd>
    <dt>Registration Number</dt>
    <dd>@Model.RegNumber</dd>
    <dt>Full Name</dt>
    <dd>@Model.FullName</dd>
    <dt>Programme</dt>
    <dd>@Model.Programme</dd>
</dl>
```

The `@model Student` directive tells Razor the type of `Model`. The view only reads properties and emits HTML. It does not query data or make business decisions.

4. Routing configuration — `Program.cs`

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Students}/{action=Index}/{id?}");
```

This conventional route is what makes `/Students/Details/7` resolve to `StudentsController.Details(7)`. The `id?` segment is optional; when present it is bound to the action parameter.

Running the demo

1. Ensure the .NET 10 SDK is installed (`dotnet --version`).
2. Open a terminal in the project folder.
3. Run `dotnet run`.
4. Open the URL printed in the terminal, then navigate to `/Students/Details/7`.
5. The page should display the details of the student with Id 7.

4 (d) Why logic must not live in the View

Placing data access and business logic directly inside a View is considered poor design for the following reasons:

- **Separation of Concerns is broken.** The view's responsibility is to present data. Retrieving data, enforcing rules and making decisions belong in the controller and model (or in services called by the controller).

When a view performs these tasks, the boundary between layers disappears and the codebase becomes harder to navigate.

- **Testing becomes difficult.** Controller actions and plain model/service classes can be unit tested with ordinary testing frameworks. Razor views are awkward to test in isolation; verifying business behaviour through rendered HTML is slow and brittle.
- **Reuse is limited.** The same validation rule or query may be needed by several screens or by an API endpoint. Logic buried inside a `.cshtml` file cannot be called from elsewhere without duplication or awkward workarounds.
- **Maintenance cost rises.** User-interface changes and business-rule changes become entangled. A developer who only wants to adjust layout risks breaking data access code, and vice versa.
- **Security and consistency suffer.** Authorisation checks, input validation and error handling are easy to omit when every view contains its own ad-hoc data code. Centralising these concerns in the controller/service layer makes consistent enforcement possible.

The recommended practice is therefore: the controller (with the help of services) loads and processes data; the view receives a ready model and focuses solely on rendering it.